



Programmieren I Abschlussprojekt

BA-MECH-26

E-Bike Simulation

Ilian Martic, 52506961

Lukas Steiner, 52506983

Inhaltsverzeichnis

I.	Einleitung	3
II.	Projektorganisation und Versionskontrolle	3
III.	Softwarearchitektur	3
IV.	Methodik und Implementierung	4
IV.1.	Physikalisches Umgebungsmodell: Dynamische Luftdichte	4
IV.2.	Batteriesimulation und thermisches Verhalten	5
IV.3.	Datenverarbeitung und Export	5
V.	Benutzeroberfläche und Simulationsablauf	6
V.1.	Integration lokaler Wetterdaten	8
V.2.	Physikalische Modellierung des Rollwiderstands	9
VI.	Testing und Qualitätssicherung	9
VII.	Fazit und Ausblick	10

I. Einleitung

Ziel dieses Projektes war die Entwicklung eines modularen E-Bike-Simulators in Python im Rahmen der Lehrveranstaltung Programmieren I. Wir haben ein Programm entworfen, das reale Streckendaten einliest und das physikalische Verhalten des Fahrrads sowie des Akkus entlang dieser Route berechnet.

Dabei konnten wir das Basissystem um praxisnahe Zusatzfunktionen erweitern. Neben einer erweiterten Fahr- und Umgebungsphysik (wie der Berücksichtigung des Rollwiderstands und der höhenabhängigen Luftdichte) wurden ein thermisches Batteriemodell mit Leistungskopplung, eine automatisierte GPS-Datenvalidierung sowie eine optionale Integration von Wetterdaten umgesetzt. Die Bedienung und Auswertung wurde zudem flexibel über eine grafische Benutzeroberfläche (GUI) sowie automatisierte Text-Reports realisiert.

Um die Softwarearchitektur sauber und wartbar zu halten, haben wir das Projekt strikt objektorientiert aufgebaut und die Berechnungslogik von der Ein- und Ausgabe getrennt. Der fertige Simulator kann wahlweise über ein Kommandozeilen-Interface (CLI) oder eine grafische Benutzeroberfläche (GUI) bedient werden. Abschließend werden die berechneten Daten in Diagrammen visualisiert und über einen Report-Generator als Textdatei gesichert. Zur Validierung der physikalischen Berechnungen und des Programmablaufs wurden zudem Unit- und Integrationstests integriert.

II. Projektorganisation und Versionskontrolle

Um eine strukturierte und parallele Zusammenarbeit im Team zu gewährleisten, wurde das Projekt von Beginn an über das Versionskontrollsystem Git verwaltet und in einem zentralen GitHub-Repository gehostet. Dies ermöglichte eine klare Aufteilung der Programmieraufgaben und eine transparente Nachverfolgung aller Code-Änderungen.

Die Aufgabenplanung und Fehlerbehebung erfolgte iterativ über das Issue-Tracking-System von GitHub. Jedes neue Feature und jeder Bug – wie etwa die Implementierung der barometrischen Höhenformel (Issue #44) oder die finale Dokumentation (Issue #18) – wurde als separate Issue erfasst. Die eigentliche Programmierung fand in isolierten Entwicklungszweigen (Branches) statt.

Nach erfolgreichem lokalen Testing wurden die Änderungen über Pull Requests in den Hauptzweig (Main-Branch) überführt. Diese strikte Vorgehensweise verhinderte Code-Konflikte zwischen den Teammitgliedern und stellte sicher, dass die Code-Basis auch bei wachsender Komplexität des Simulators stets lauffähig blieb.

III. Softwarearchitektur

Das Projekt folgt strikt dem Paradigma der objektorientierten Programmierung (OOP). Die zentrale Komponente der Kernlogik bildet die Klasse `Simulator`, welche den zeitdiskreten Ablauf der Berechnung steuert. Sie führt die Modelle des Fahrrads (`EBike`) und der physikalischen Strecke (`Route`) zusammen.

Um das System erweiterbar zu halten, ist das `EBike` modular aufgebaut. Es aggregiert die physischen Bauteile und Eigenschaften: Eine `EBikeConfig` (für Parameter wie die Gesamtmasse), einen `Motor` (für Leistungs- und Wirkungsgradberechnungen) sowie eine `Battery`. Letztere ist als abstrakte Basisklasse implementiert. Dies ermöglicht es, verschiedene Akku-Technologien (wie z. B. spezifische LiPo-Modelle) mit ihren individuellen Entlade- und Temperaturkurven flexibel auszutauschen.

Die Umgebungsdatenbank wird über den `RouteAnalyzer` verarbeitet und in einer Liste von `RoutePoint`-Objekten strukturiert. Jeder Punkt kapselt die lokalen Geodaten (Koordinaten, Höhe) sowie die Wetterbedin-

gungen (Windgeschwindigkeit, Temperatur).

Ein wesentliches Architekturziel war die Trennung von Berechnungslogik und Datendarstellung. Die Kernklassen im `src`-Verzeichnis führen ausschließlich die Simulation durch. Die visuelle Aufbereitung der Ergebnisse wird vollständig in separate Module wie den `Plotter` (für Diagramme) und den `ReportGenerator` ausgelagert, welche durch die jeweiligen Einstiegspunkte (GUI oder CLI) angesteuert werden.

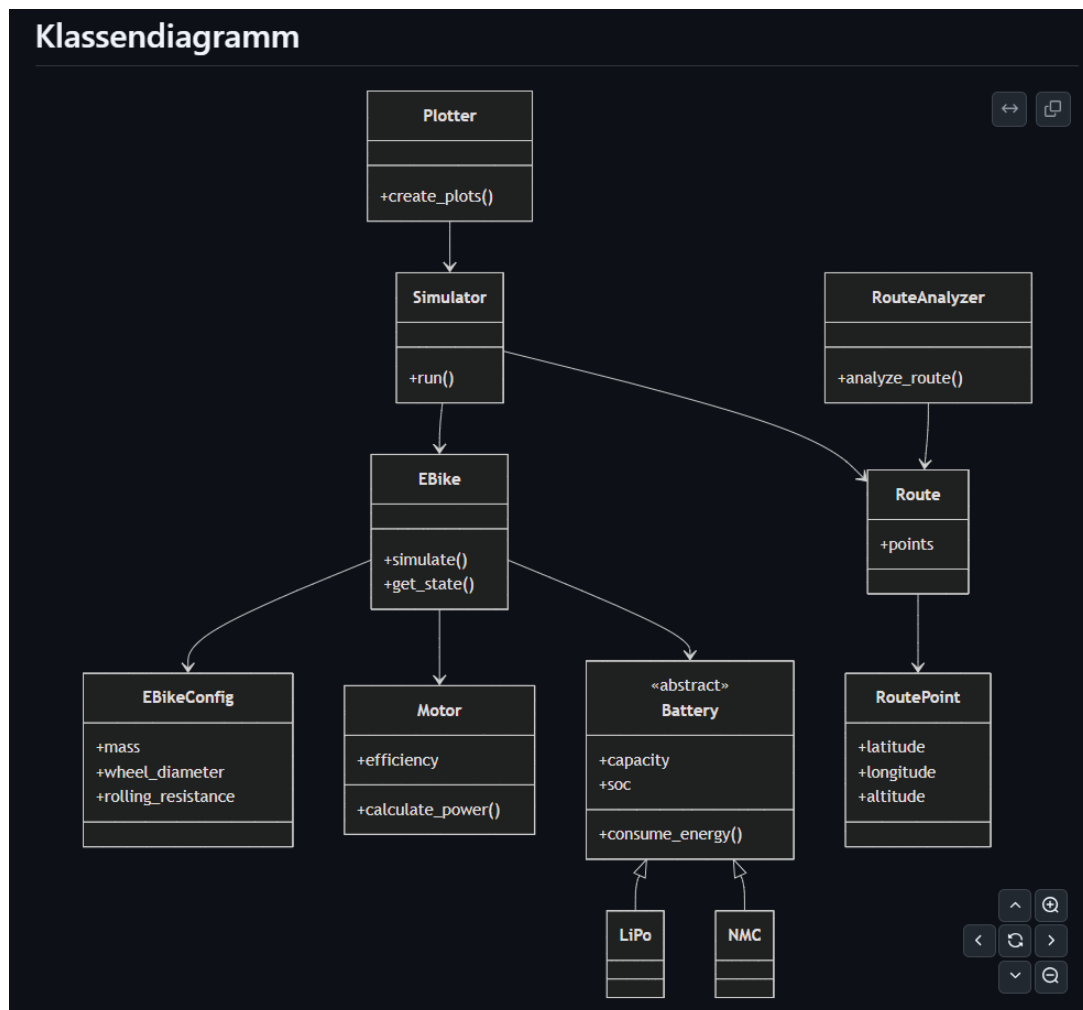


Abbildung 1.: UML-Klassendiagramm des E-Bike-Simulators

IV. Methodik und Implementierung

In diesem Kapitel werden die wichtigsten Funktionen des Simulators detailliert betrachtet. Der Fokus liegt auf der physikalischen Modellierung der Umgebung und der Batterie sowie der informationstechnischen Verarbeitung der Simulationsdaten.

IV.1. Physikalisches Umgebungsmodell: Dynamische Luftdichte

Zur Erhöhung der Simulationsgenauigkeit wurde eine dynamische Berechnung der Luftdichte in Abhängigkeit der aktuellen Seehöhe implementiert. Anstatt eines statischen Standardwertes wird die barometrische Höhenformel angewendet, um die abnehmende Dichte der Atmosphäre auf Gebirgspässen und Steigungen korrekt abzubilden. Dies wirkt sich direkt auf den berechneten Luftwiderstand aus und führt insbesondere

bei Routen mit großen Höhendifferenzen zu realistischeren Leistungswerten des Motors. Die Umsetzung in der Simulationsschleife übergibt die aktuelle Höhe dynamisch an die Schrittfunktion des Fahrradmodells.

```

1 def air_drag(self, velocity: float, wind_speed: float, elevation: float = 0.0,
    ambient_temperature: float = 15.0) -> float:
2     """Berechnet den Luftwiderstand unter Berücksichtigung von Wind, Höhe und Temperatur
    """
3     # Dynamische Luftdichte berechnen
4     air_density = self._calculate_air_density(elevation, ambient_temperature)
5
6     rel_speed = velocity - wind_speed
7
8     #  $F_w = 0.5 \cdot \rho \cdot c_{w\_A} \cdot v_{rel}^2$ 
9     return 0.5 * air_density * self.config.c_w_a * rel_speed**2

```

Listing 1: Berechnung des Luftwiderstands mit dynamischer Luftdichte

IV.2. Batteriesimulation und thermisches Verhalten

Das Batteriemodell berechnet die Temperaturentwicklung der Zellen während der Fahrt. Dabei haben wir zwei Effekte kombiniert: die Erwärmung durch den fließenden Strom (Joulesche Wärme) und die passive Abkühlung durch die Umgebungsluft (Newtonsches Abkühlungsgesetz).

Da die Simulation die Zeit (dt) zwischen den einzelnen Routenpunkten teils in großen diskreten Blöcken (mehrere Minuten am Stück) verarbeitet, stieß eine einfache lineare Integration mit den ursprünglich theoretisch hergeleiteten Vorfaktoren an numerische Grenzen. Dies führte in frühen Testläufen zu unrealistischen Temperaturspitzen, da sich die Werte bei großen dt -Sprüngen aufschaukelten.

Um die thermische Trägheit eines massiven E-Bike-Akkus auch bei großen Zeitschritten numerisch stabil abzubilden, wurden die Konstanten empirisch an das Modell angepasst. Für die Erwärmung (k_{heat}) hat sich ein Wert von 0,00001 als realistisch erwiesen, für die Abkühlung ($k_{cooling}$) ein Wert von 0,0001. Zusätzlich implementiert das Modell eine Plausibilitätsprüfung, die verhindert, dass die passive Abkühlung den Akku physikalisch inkorrekt unter die Umgebungstemperatur fallen lässt.

```

1 def update_temperature(self, current: float, ambient_temperature: float, dt: float) ->
    None:
2     k_heat = 0.00001
3     k_cooling = 0.0001
4
5     heating = k_heat * current**2
6     cooling = k_cooling * (self.temperature - ambient_temperature)
7
8     self.temperature += (heating - cooling) * dt
9
10    # Plausibilitaetspruefung: Abkuehlung nicht unter Umgebungstemperatur
11    if current == 0.0 and self.temperature < ambient_temperature:
12        self.temperature = ambient_temperature

```

Listing 2: Implementierung des thermischen Batteriemodells

IV.3. Datenverarbeitung und Export

Für die Auswertung der simulierten Fahrten wurde ein eigener ReportGenerator implementiert. Der Architektur-Gedanke dabei war, die eigentliche Berechnung strikt von der Datenausgabe zu trennen.

Sobald die Simulation abgeschlossen ist, greift das Modul auf das gesammelte Daten-Dictionary der Benutzeroberfläche zu und exportiert die zentralen Kennzahlen (Gesamtstrecke, verbrauchte Energie, Maximaltemperaturen) parallel in zwei Formaten: als Textdatei für schnelles Logging und einmal als formatiertes PDF-Dokument. Für die Erstellung des PDFs wurde die externe Bibliothek `fpdf2` eingebunden.

Dadurch werden die Ergebnisse jedes Durchlaufs automatisch gesichert und können im Nachhinein ausgewertet oder weitergegeben werden, ohne sich zwingend durch die Diagramme der GUI klicken zu müssen.

```
1 from fpdf import FPDF
2 import os
3
4 class ReportGenerator:
5     @staticmethod
6     def generate_txt_report(filepath: str, stats: dict) -> str:
7         # ... (Bestehende Generierung der Textdatei) ...
8         pass
9
10    @staticmethod
11    def generate_pdf_report(filepath: str, stats: dict) -> str:
12        """Generiert einen formatierten PDF-Report."""
13        if not filepath.endswith('.pdf'):
14            filepath = filepath.split('.')[0] + '.pdf'
15
16        os.makedirs(os.path.dirname(os.path.abspath(filepath)), exist_ok=True)
17
18        pdf = FPDF(orientation="P", unit="mm", format="A4")
19        pdf.add_page()
20
21        # ... (Formatierung und Einfuegen der GUI-Daten in das Dokument) ...
22
23        pdf.output(filepath)
24        return filepath
```

Listing 3: Auszug aus der erweiterten Report-Generierung mit PDF-Support

V. Benutzeroberfläche und Simulationsablauf

Damit der Simulator auch ohne Programmierkenntnisse genutzt werden kann, haben wir neben der Kommandozeile (CLI) eine grafische Benutzeroberfläche (GUI) entwickelt.

Der typische Simulationsablauf beginnt mit dem Einlesen der topografischen Daten. Der Nutzer wählt über die GUI eine Routen-Datei (im CSV-Format) aus und kann Startparameter wie das Gesamtgewicht des Systems und den initialen Ladezustand des Akkus (SoC) festlegen.

Während der Simulation berechnet die Logikschicht die physikalischen Zustände für jeden Routenpunkt. Nach Abschluss des Durchlaufs generiert die Anwendung automatisch verschiedene Diagramme über das Plotter-Modul. Diese visualisieren beispielsweise das Streckenprofil im Verhältnis zum Energieverbrauch oder den Temperaturverlauf des Akkus. Parallel dazu wird im Hintergrund der bereits erwähnte Text-Report generiert, sodass der Nutzer am Ende sowohl eine schnelle grafische Übersicht als auch eine detaillierte textbasierte Zusammenfassung der Fahrt erhält.

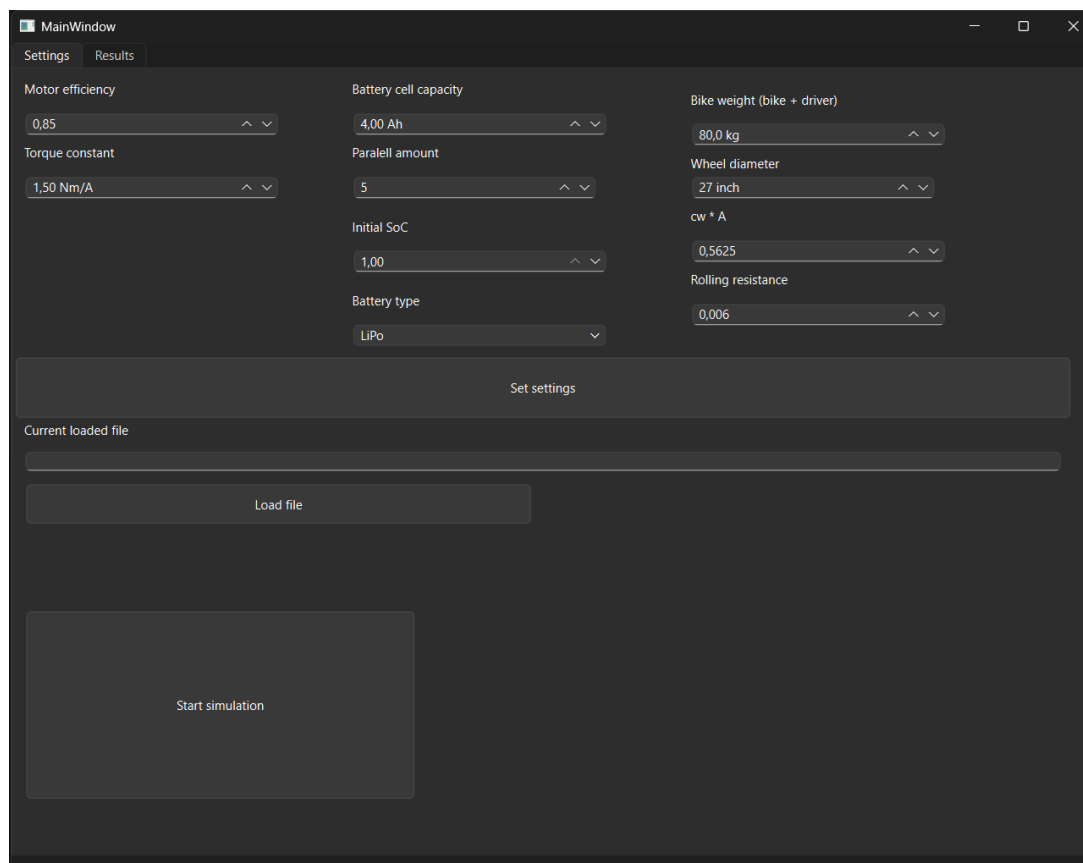


Abbildung 2.: Grafische Benutzeroberfläche des E-Bike-Simulators

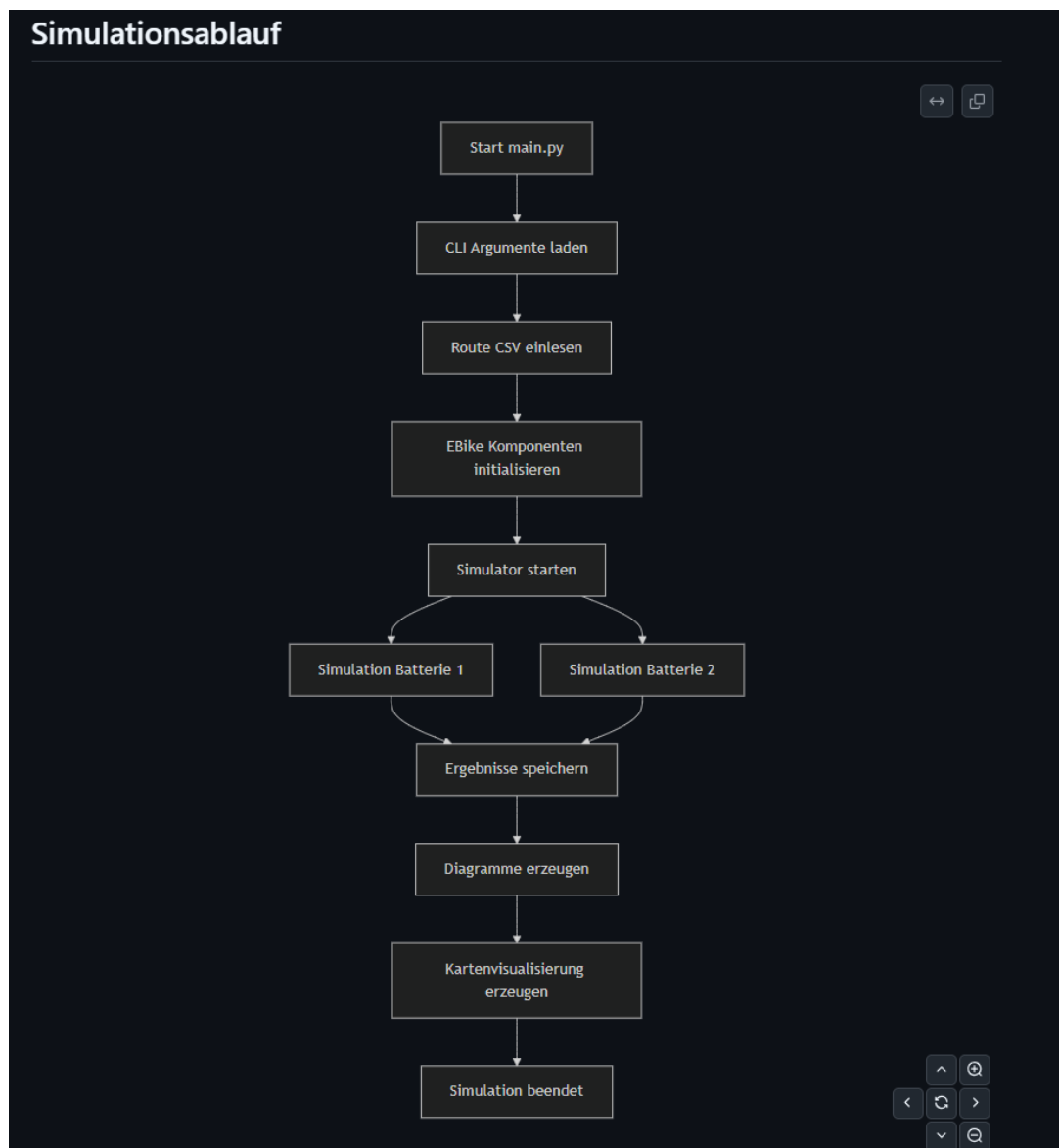


Abbildung 3.: UML-Klassendiagramm des Simulationsablaufs

V.1. Integration lokaler Wetterdaten

Um die Simulation realistischer zu machen ruft der Simulator historische Wetterdaten über die Open-Meteo-API ab. Um die Anzahl der Netzwerkanfragen zu minimieren und die Performance zu gewährleisten, werden die abgerufenen stündlichen Daten (Temperatur, Windgeschwindigkeit und -richtung) pro Tag in einem Dictionary zwischengespeichert (Caching). Da die Zeitstempel der Routenpunkte in der Regel zwischen den vollen Stunden liegen, wurde eine lineare Interpolation implementiert. Diese berechnet den exakten Wetterzustand für den jeweiligen Wegpunkt zeitanteilig auf die Sekunde genau.

```

1 # Berechnung des zeitlichen Faktors zwischen zwei vollen Stunden
2 passed_seconds = (point.time - times[before]).total_seconds()
3 factor = passed_seconds / total_seconds
4
5 # Lineare Interpolation der Temperatur auf den exakten Zeitstempel
6 point.temperature = WeatherLoader.interpolate(
7 data["hourly"]["temperature_2m"][before],
8 data["hourly"]["temperature_2m"][after],
9 factor

```


Listing 4: Lineare Interpolation der Wetterdaten**V.2. Physikalische Modellierung des Rollwiderstands**

Neben dem Luftwiderstand und der Hangabtriebskraft muss der Motor zusätzlich den Rollwiderstand überwinden, welcher durch die Verformung der Reifen bestimmt wird. Die Berechnung erfolgt über die physikalische Grundgleichung:

$$F_{roll} = c_R \cdot m \cdot g \cdot \cos(\alpha)$$

Der benötigte Steigungswinkel α wird dabei über den Arkustangens aus der prozentualen Steigung (Slope) des aktuellen Routenabschnitts hergeleitet ($\alpha = \arctan(\text{slope})$). Die daraus resultierende Kraft fließt anschließend direkt in die Gesamtkraftbilanz des E-Bikes ein, aus der die benötigte mechanische Motorleistung abgeleitet wird.

```

1 def rolling_resistance(self, slope: float = 0.0) -> float:
2     # Aus Steigung den Winkel berechnen
3     alpha = math.atan(slope)
4     return self.config.rolling_resistance * self.config.mass * 9.81 * math.cos(alpha)
5
6 def required_power(self, velocity: float, slope: float, elevation: float = 0.0,
7     ambient_temperature: float = 15.0):
8     F_air = self.air_drag(velocity, 0.0, elevation, ambient_temperature)
9     F_slope = self.slope_force(slope)
10    F_rolling_resistance = self.rolling_resistance(slope)
11
12    total_force = F_air + F_slope + F_rolling_resistance
13    return total_force * velocity

```

Listing 5: Berechnung des Rollwiderstands und der Gesamtkraft**VI. Testing und Qualitätssicherung**

Um sicherzustellen, dass die physikalischen Modelle korrekt rechnen und das Programm bei fehlerhaften Eingaben nicht abstürzt, haben wir den Code mit dem in Python integrierten `unittest`-Framework abgesichert.

Der Fokus lag dabei auf den Kernkomponenten: dem Motor und der Batterie. Wir haben gezielt Grenzfälle (Edge-Cases) getestet. Beispielsweise muss die Simulation korrekt reagieren, wenn der Akku während der Fahrt einen Ladezustand von 0% erreicht (der Motor darf dann keine Leistung mehr erbringen), oder wenn das E-Bike ein starkes Gefälle hinabfährt und der Leistungsbedarf auf null sinkt.

Zusätzlich zu diesen Unit-Tests haben wir Integrationstests geschrieben, die den kompletten Ablauf eines Simulationsschritts (inklusive Luftwiderstand, Rollwiderstand und Thermik) prüfen. Alle Tests können über einen zentralen Konsolenbefehl ausgeführt werden, was uns während der Entwicklung geholfen hat, Bugs frühzeitig zu erkennen, bevor Änderungen in den Main-Branch gemergt wurden.

VII. Fazit und Ausblick

Im Rahmen dieser Projektarbeit wurde ein funktionsfähiger, modularer E-Bike-Simulator in Python entwickelt. Die vorgegebenen Minimalanforderungen – wie die objektorientierte Modellierung eines Fahrrads und die Berechnung des grundlegenden Energieverbrauchs – wurden dabei nicht nur vollständig erfüllt, sondern zudem durch verschiedene Zusatzfunktionen erweitert.

Um die physikalische Genauigkeit und die softwaretechnische Qualität der Simulation zu verbessern, wurden folgende spezifische Erweiterungen in das System integriert:

- **Erweiterte Umgebungs- und Fahrphysik:** Direkte Integration von Wetterdaten, dynamische Bestimmung der Luftdichte aus lokaler Temperatur und aktueller Seehöhe sowie die physikalische Simulation des Rollwiderstands.
- **Komplexes Batteriemodell:** Kontinuierliche Simulation der Akkutemperatur inklusive der Rückkopplung dieses thermischen Verhaltens auf die tatsächliche Leistung und Effizienz des Akkus.
- **Datenvalidierung und -export:** Implementierung einer automatisierten Validierung der eingelesenen GPS-Routendaten zur Fehlerprävention sowie das Generieren eines strukturierten Text-Reports zur dauerhaften Sicherung der Simulationsergebnisse.
- **Flexible Benutzerinteraktion:** Vollständige Steuerbarkeit der Software wahlweise über eine entwickelte grafische Benutzeroberfläche (GUI) oder durch die Übergabe von Startargumenten (CLI) beim Aufruf der Python-Datei.

Für zukünftige Entwicklungszyklen bietet diese Code-Basis ein hervorragendes Fundament. Denkbare Ausbaustufen wären die Einbindung von Modellen zur Batterie-Degradation. Zusammenfassend belegt das Projekt, dass neben der reinen algorithmischen Umsetzung ein hoher Wert auf physikalische Plausibilität, saubere Architektur und eine professionelle Entwicklungsstruktur gelegt wurde.